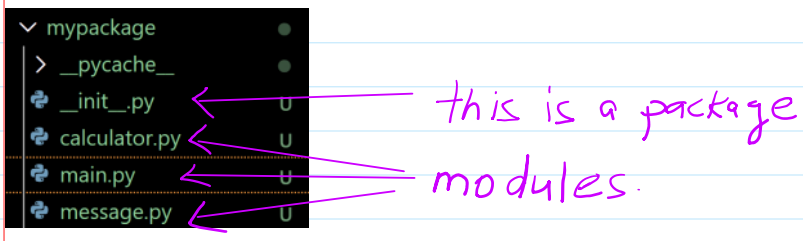
# Module, Package, Library in Python

03 June 2026 06:32

## Module in Python

A **module** is simply a Python file (.py) that contains variables, functions, classes or executable code.

A module helps organize code and promotes code reuse.




mypackage > message.py > greet

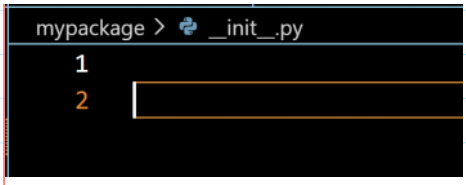
```
1  def greet(name):
2      print("Hello", name)
```

mypackage > calculator.py > sub

```
1
2  pi = 3.14159
3
4  def add(a, b):
5      return a+b
6
7  def sub(a, b):
8      return a - b
```

mypackage > main.py

```
1  import calculator
2  import message
3
4  print(calculator.add(3, 4))
5  print(calculator.sub(6, 2))
6  print(calculator.pi)
7
8  message.greet('Vannya')
9
```




**Importing Specific functions**

```
from calculator import sub
print(sub(7, 4))
```

**Importing Multiple functions**

```
from calculator import sub, add
print(sub(7, 4))
print(add(6, 8))
```

**Importing everything**

```
from calculator import *
print(sub(7, 4))
print(add(6, 8))
print(pi)
```

**Output:**

**3**
**14**
**3.14159**

Use modules to write similar functions

If you use modules concept the code maintainence will be easy.

**Why use Modules?**

- **Code reusability**
- **Better organization**
- **Easier maintenance**
- **Separation of concerns**
- **Simplifies debugging**

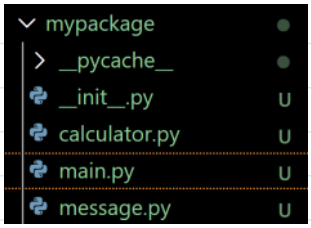
==========

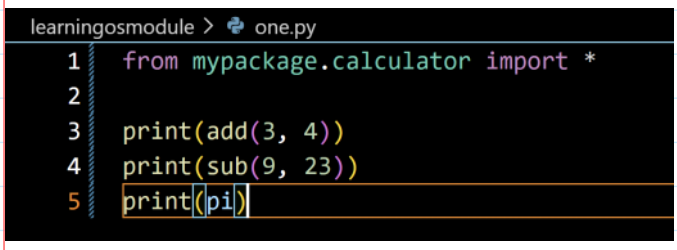
**Package in Python**

A **package** is a collection of related modules organized inside a directory.

Think of:

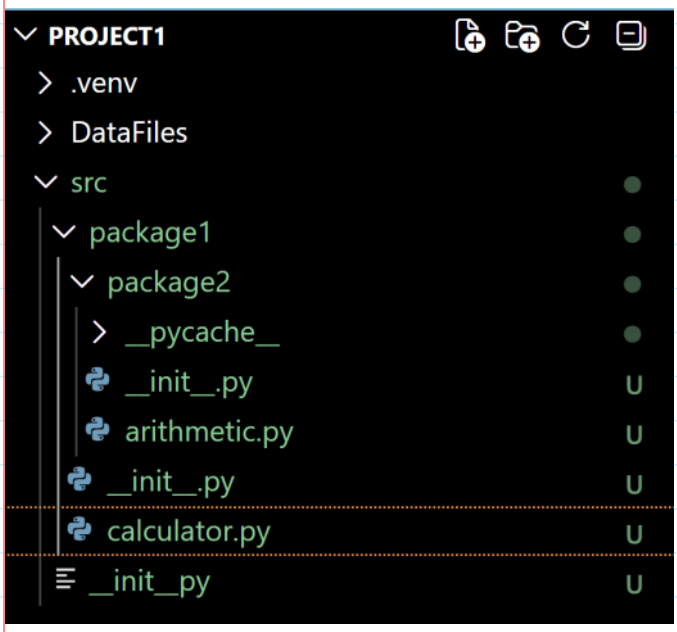
Module = Single Python File
Package = Folder containing Python modules




**To run or execute this code we need to run command on the terminal:**

**python -m learningosmodule.one**



src > package1 > calculator.py > ...

```python
import package2.arithmetic as arith

while True:
    op = input('Press x or X to exit or enter to continue: ')
    if op in ('x', 'X'):
        print('Good bye, \nThank you')
        exit()
    x = int(input('Enter first number: '))
    y = int(input('Enter second number: '))
    op = input("+,-, *, //, %, ^ : ")

    match op:
        case '+':
            print(f'sum = {arith.add(x, y)}')
        case '-':
            print(f'difference = {arith.sub(x, y)}')
        case '*':
            print(f'product = {arith.multiply(x, y)}')
        case '//':
            print(f'Quotient = {arith.quotient(x, y)}')
        case '%':
            print(f'remainder = {arith.remainder(x, y)}')
        case '^':
            print(f'power = {arith.power(x, y)}')
        case _:
            print('Wrong option selected!')

```

src > package1 > package2 > arithmetic.py > power

```python
def add(a, b):
    return a + b


def sub(a, b):
    return a - b


def multiply(a, b):
    return a * b


def quotient(a, b):
    return a // b


def remainder(a, b):
    return a % b

def power(a, b):
    return a ** b
```

**Library in Python**

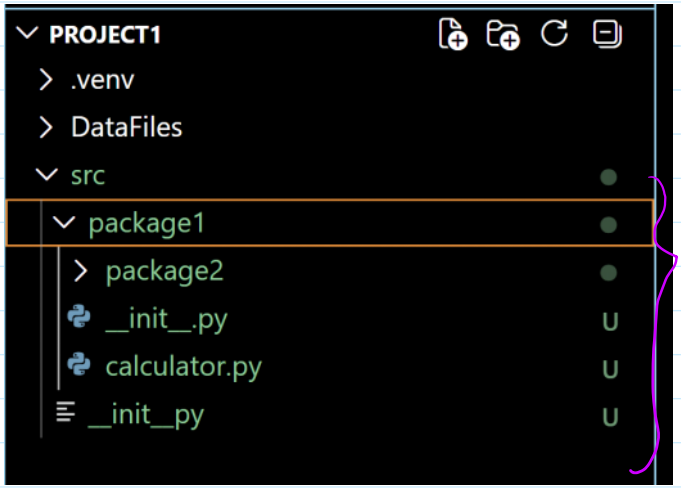
A library is a collection of modules and packages that provide specific functionality.

Libraries are usually created by Python developers or third-party

organizations.

Example of Libraries

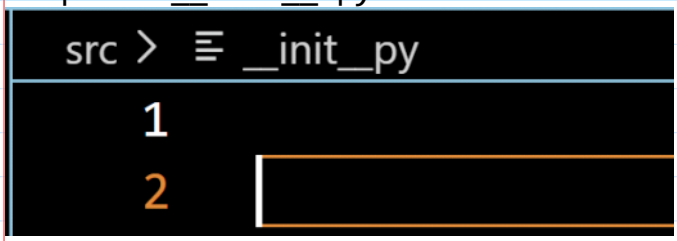
| Library | Purpose |
|---|---|
| math | Mathematical function |
| random | Random number generations |
| os | Operating system operations |
| datetime | Date and time handling |
| numpy | Numerical computing |
| pandas | Data analysis |
| matplotlib | Data visualization |
| tkinter | GUI application |




Library
package
package1
package2

**What is __init__.py?**
__init__.py is a special file that tells Python that a directory should be tread as a package.

Simplest __init__.py:




Even an empty file makes the folder a package.

**Why use __init__.py?**

**1. Marks Directory or folder as a package.**

2. Execute Package Initialization code.

a.

src > package1 > package2 > __init__.py

```
1   print('package2 is loaded')
```

Output:

```
package2 is loaded
```

Home work:

1. Practice the above concept.
2. Divide the student management code into multiple .py files.